

数据结构历年真题分类

第一章 绪论

- (一) 数据结构的基本概念
- (二) 算法的基本概念

第二章 线性表

- (一) 线性表的基本概念
- (二) 线性表的实现
 - 顺序存储
 - 链式存储
- (三) 线性表的应用

第三章 栈、队列和数组

- (一) 栈和队列的基本概念
- (二) 栈和队列的顺序存储结构
- (三) 栈和队列的链式存储结构
- (四) 多维数组的存储
- (五) 特殊矩阵的压缩存储
- (六) 栈、队列和数组的应用

第四章 树与二叉树

- (一) 树的基本概念
- (二) 二叉树
 - 二叉树的定义及其主要特征
 - 二叉树的顺序存储结构和链式存储结构
 - 二叉树的遍历
 - 线索二叉树的基本概念和构造
- (三) 树、森林
 - 树的存储结构
 - 森林与二叉树的转换
 - 树和森林的遍历
- (四) 树与二叉树的应用

1. 哈夫曼(Huffman)树和哈夫曼编码

2. 并查集及其应用

3. 堆及其应用

第五章 图

(一)图的基本概念

(二)图的存储及基本操作

(三)图的遍历

(四)图的基本应用

最小(代价)生成树

最短路径

拓扑排序

关键路径

第六章 查找

(一)查找的基本概念

(二)顺序查找法

(三)分块查找法

(四)折半查找法

(五)树型查找

二叉搜索树

平衡二叉树

红黑树

(六)B树及其基本操作、B+树的基本概念

(七)散列(Hash)表

(八)字符串模式匹配

(九)查找算法的分析及应用

第七章 排序

(一)排序的基本概念

(二)直接插入排序

(三)折半插入排序

(四)起泡排序(bubble sort)

(五)简单选择排序

(六)希尔排序(shell sort)

(七)快速排序

(八)堆排序

(九)二路归并排序(merge sort)

(十)基数排序

(十一)外部排序

(十二)排序算法的分析和应用

答案见: <https://www.yuque.com/yuqueyonghunhc3kb/darkmoon?#> 《408笔记试看》(答案免费, 持续更新中)

第一章 绪论

(一) 数据结构的基本概念

(二) 算法的基本概念

【2011-01】设 n 是描述问题规模的非负整数, 下面程序片段的时间复杂度是 ()

```
x = 2;
while (x < n / 2)
    x = 2 * x;
```

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

【2012-01】求整数 $n(n \geq 0)$ 阶乘的算法如下, 其时间复杂度是 ()。

```
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

【2013-01】已知两个长度分别为 m 和 n 的升序链表, 若将它们合并为长度为 $m+n$ 的降序链表, 则最坏情况下的时间复杂度是 ()。

A. $O(n)$
B. $O(mn)$
C. $O(\min(m, n))$
D. $O(\max(m, n))$

【2014-01】下列程序段的时间复杂度是 ()

```
count = 0;
for(k = 1; k <= n; k *= 2)
    for(j = 1; j <= n; j++)
        count++;
```

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

【2017-01】下列函数的时间复杂度是 ()

```
int func(int n) {
    int i = 0, sum = 0;
    while(sum < n) sum += ++i;
    return i;
}
```

A. $O(\log n)$ B. $O(n^{1/2})$ C. $O(n)$ D. $O(n \log n)$

【2019-01】设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是 ()

```
x = 0;
while (n >= (x + 1) * (x + 1))
    x = x + 1;
```

A. $O(\log_{10} n)$ B. $O(n^{1/2})$ C. $O(n)$ D. $O(n^2)$

【2022-01】下列程序段的时间复杂度是 ()

```
int sum = 0; for (int i = 1; i < n; i *= 2)
    for (int j = 0; j < i; j++)
        sum++;
```

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

【2023-01】下列对顺序存储的有序表（长度为 n ）实现给定操作的算法中，平均时间复杂度为 $O(1)$ 的是 ()。

- A. 查找包含指定值的元素的位置
- B. 插入包含指定值元素的算法
- C. 删除第 i ($1 \leq i \leq n$) 个元素的算法
- D. 获取第 i ($1 \leq i \leq n$) 个元素的算法

第二章 线性表

(一)线性表的基本概念

(二)线性表的实现

顺序存储

1. 【2023-01】 下列对顺序存储的有序表（长度为 n ）实现给定操作的算法中，平均时间复杂度为 $O(1)$ 的是（ ）。
- A. 查找包含指定值的元素的位置
 - B. 插入包含指定值元素的算法
 - C. 删除第 $i(1 \leq i \leq n)$ 个元素的算法
 - D. 获取第 $i(1 \leq i \leq n)$ 个元素的算法

链式存储

2. 【2013-01】 已知两个长度分别为 m 和 n 的升序链表，若将它们合并为长度为 $m+n$ 的降序链表，则最坏情况下的时间复杂度是（ ）。
- A. $O(n)$
 - B. $O(mn)$
 - C. $O(\min(m,n))$
 - D. $O(\max(m,n))$
3. 【2016-01】 已知表头元素为 c 的单链表在内存中的存储状态如下表所示。

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将 f 存放于 1014H 处并插入到单链表中，若 f 在逻辑上位于 a 和 e 之间，则 a, e, f 的“链接地址”依次是（ ）。

- A. 1010H, 1014H, 1004H
 - B. 1010H, 1004H, 1014H
 - C. 1014H, 1010H, 1004H
 - D. 1014H, 1004H, 1010H
4. 【2016-02】 已知一个带有表头结点的双向循环链表 L ，结点结构为 $\text{prev}|\text{data}|\text{next}$ ， prev 和 next 分别是指向其直接前驱和直接后继结点的指针。现要删除指针 p 所指的结点，正确的语句序列是（ ）。
- A. `p->next->prev = p->prev; p->prev->next = p->prev; free(p);`
 - B. `p->next->prev = p->next; p->prev->next = p->next; free(p);`
 - C. `p->next->prev = p->next; p->prev->next = p->prev; free(p);`
 - D. `p->next->prev = p->prev; p->prev->next = p->next; free(p);`
5. 【2021-01】 已知头指针 h 指向一个带头结点的非空单循环链表，结点结构为 $\text{data}|\text{next}$ ，其中 next 是指向直接后继结点的指针， p 是尾指针， q 是临时指针。现要删除该链表的第一个元素，正确的语句序列是（ ）。
- A. `h->next = h->next->next; q = h->next; free(q);`
 - B. `q = h->next; h->next = h->next->next; free(q);`

- C. `q = h->next; h->next = q->next; if (p != q) p = h; free(q);`
- D. `q = h->next; h->next = q->next; if (p == q) p = h; free(q);`

6. 【2023-02】现有非空双向链表 L，其结点结构为：

prev	data	next
------	------	------

，prev 是指向直接前驱结点的指针，next 是指向直接后继结点的指针，若要在 L 中指针 p 所指向的结点（非尾结点）之后插入指针 s 指向的新结点，则在执行了语句“`s->next = p->next; p->next = s;`”后，下列语句序列中还需要执行的是（ ）。

- A. `s->next->prev = p; s->prev = p;`
- B. `p->next->prev = s; s->prev = p;`
- C. `s->prev = s->next->prev; s->next->prev = s;`
- D. `p->next->prev = s->prev; s->next->prev = p;`

7. 【2024-01】已知带头结点的非空链表 L 的头指针为 h，结点结构为

data	next
------	------

，其中 next 是指向直接后继结点的指针。现有指针 p 和 q，若 p 指向 L 中非首且非尾的任意一个结点，则执行语句序列 `q = p->next; p->next = q->next; q->next = h->next; h->next = q;` 的结果是

- A. 在 p 所指结点后插入 q 所指结点
- B. 在 q 所指结点后插入 p 所指结点
- C. 将 p 所指结点移动到 L 的头结点之后
- D. 将 q 所指结点移动到 L 的头结点之后

8. 【2026-01】当存储空间有足够的空闲空间时，在保持表内元素顺序相对不变的情况下，下列哪些操作会必然导致产生移动次数（ ）。

- I. 表头插入一个元素
- II. 表头删除一个元素
- III. 表尾插入一个元素
- IV. 表尾删除一个元素
- A. I、II
- B. I、III
- C. II、IV
- D. III、IV

9. 【2026-02】设有一个双向链表 L，结构为

p2	p1
----	----

，头结点为 head。初始时 `head = cu`。现要将每个结点的 p2 指向 p1 指向结点的直接后继，应该进行的操作是（ ）。

- A. `while(cu!=NULL) {cu->p2=cu->p1->p1; cu=cu->p1;}`
- B. `while(cu!=NULL && cu->p2!=NULL) {cu->p2 = cu->p1->p1; cu = cu->p1;}`
- C. `while(cu!=NULL) {if(cu->p1!=NULL) {cu->p2=cu->p1->p1; cu=cu->p1;}}`
- D. `while(cu!=NULL) {if(cu->p1!=NULL) {cu->p2=cu->p1->p1;} else {cu->p2=NULL;} cu=cu->p1;}`

(三)线性表的应用

第三章 栈、队列和数组

(一) 栈和队列的基本概念

(二) 栈和队列的顺序存储结构

1. 【2009-02】 设栈 S 和队列 Q 的初始状态均为空，元素 a, b, c, d, e, f, g 依次进入栈 S 。若每个元素出栈后立即进入队列 Q ，且 7 个元素出队的顺序是 b, d, c, f, e, a, g ，则栈 S 的容量至少是 ()。
A. 1
B. 2
C. 3
D. 4
2. 【2010-01】 若元素 a, b, c, d, e, f 依次进栈，允许进栈、退栈操作交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是 ()。
A. d, c, e, b, f, a
B. c, b, d, a, e, f
C. b, c, a, e, f, d
D. a, f, e, d, c, b
3. 【2010-02】 某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作，若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是 ()。
A. b, a, c, d, e
B. d, b, a, c, e
C. d, b, c, a, e
D. e, c, b, a, d
4. 【2011-02】 元素 a, b, c, d, e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是 ()
A. 3 B. 4 C. 5 D. 6
5. 【2011-03】 已知循环队列存储在一维数组 $A[0..n-1]$ ，且队列非空时， $front$ 和 $rear$ 分别指向队头元素和队尾元素。若初始时队列为空，且要求第一个进入队列的元素存储在 $A[0]$ 处，则初始时 $front$ 和 $rear$ 的值分别是 ()。
A. 0,0
B. 0,n-1
C. n-1,0
D. n-1,n-1
6. 【2013-02】 一个栈的入栈序列为 $1, 2, 3, \dots, n$ ，其出栈序列是 $p_1, p_2, p_3, \dots, p_n$ ，若 $p_2 = 3$ ，则 p_3 可能取值的个数是 ()。

- A. $n-3$
- B. $n-2$
- C. $n-1$
- D. 无法确定

7. 【2018-01】若栈 S1 中保存整数，栈 S2 中保存运算符，函数 F() 依次执行下述各步操作：

- a. 从 S1 中依次弹出两个操作数 a 和 b；
- b. 从 S2 中弹出一个运算符 op；
- c. 执行相应的运算 $b \text{ op } a$ ；
- d. 将运算结果压入 S1。

假定 S1 中的操作数依次是 5, 8, 3, 2 (2 在栈顶)，S2 中的运算符依次是 $\times, -, +$ (+ 在栈顶)。调用 3 次 F() 后，S1 栈顶保存的值是 ()。

- A. -15
- B. 15
- C. -20
- D. 20

8. 【2018-02】现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6 (1 在队头)，S 为空。若仅允许下列 3 种操作：

- ① 出队并输出出队元素；
- ② 出队并将出队元素入栈；
- ③ 出栈并输出出栈元素；

则不能得到的输出序列是 ()。

- A. 1, 2, 5, 6, 4, 3
- B. 2, 3, 4, 5, 6, 1
- C. 3, 4, 5, 6, 1, 2
- D. 6, 5, 4, 3, 2, 1

9. 【2020-02】对空栈 S 进行 Push 和 Pop 操作，入栈序列为 a, b, c, d, e，经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是 ()。

- A. b, a, c
- B. b, a, e
- C. b, c, a
- D. b, c, e

10. 【2014-03】循环队列放在一维数组 $A[0..M-1]$ 中，end1 指向队头元素，end2 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 $M-1$ 个元素。初始时空。下列判断队空和队满的条件中，正确的是 ()。

- A. 队空： $\text{end1} == \text{end2}$ ；队满： $\text{end1} == (\text{end2} + 1) \bmod M$

- B. 队空: $\text{end1} == \text{end2}$; 队满: $\text{end2} == (\text{end1} + 1) \bmod (M - 1)$
C. 队空: $\text{end1} == (\text{end1} + 1) \bmod M$; 队满: $\text{end1} == (\text{end2} + 1) \bmod M$
D. 队空: $\text{end1} == (\text{end2} + 1) \bmod M$; 队满: $\text{end2} == (\text{end1} + 1) \bmod (M - 1)$

11. 【2021-02】已知初始为空的队列Q的一端仅能进行入队操作, 另外一端既能进行入队操作又能进行出队操作。若Q的入队序列是1, 2, 3, 4, 5, 则不能得到的出队序列是 ()。
- A. 5, 4, 3, 1, 2
B. 5, 3, 1, 2, 4
C. 4, 2, 1, 3, 5
D. 4, 1, 3, 2, 5
12. 【2022-02】给定有限符号集 S , in 和 out 均为 S 中所有元素的任意排列。对于初始为空的栈ST, 下列叙述中, 正确的是 ()。
- A. 若 in 是ST的入栈序列, 则不能判断 out 是否为其可能的出栈序列
B. 若 out 是ST的出栈序列, 则不能判断 in 是否为其可能的入栈序列
C. 若 in 是ST的入栈序列, out 是对应 in 的出栈序列, 则 in 与 out 一定不同
D. 若 in 是ST的入栈序列, out 是对应 in 的出栈序列, 则 in 与 out 可能互为倒序

(三) 栈和队列的链式存储结构

(四) 多维数组的存储

13. 【2023-03】若采用三元组表存储结构存储系数矩阵 M 。则除三元组外, 下列数据中还需要保存的是 ()。
- I. M 的行数
II M 中包含非零元素的行数
III. M 的列数
IV. M 中包含非零元素的列数
- A. 仅 I 和 III
B. 仅 I 和 IV
C. 仅 II 和 IV
D. I、II、III 和 IV

(五) 特殊矩阵的压缩存储

14. 【2016-04】有一个100阶的三对角矩阵 M , 其元素 $m_{\{i,j\}}(1 \leq i \leq 100, 1 \leq j \leq 100)$ 按行优先依次压缩存入下标从0开始的一维数组 N 中。元素 $m_{\{30,30\}}$ 在数组 N 中的下标是 ()。
- A. 86
B. 87
C. 88
D. 89
15. 【2017-03】适用于压缩存储稀疏矩阵的两种存储结构是 ()。

- A. 三元组表和十字链表
- B. 三元组表和邻接矩阵
- C. 十字链表和二叉链表
- D. 邻接矩阵和十字链表

16. 【2018-03】 设有一个 12×12 的对称矩阵 M ，将其上三角部分的元素 $m_{\{ij\}} (1 \leq i \leq j \leq 12)$ 按行优先存入C语言的一维数组N中，元素 $m_{\{6,6\}}$ 在N中的下标是（ ）。

- A. 50
- B. 51
- C. 55
- D. 66

17. 【2020-01】 将一个 10×10 的对称矩阵 M 的上三角部分的元素 $m_{\{i,j\}} (1 \leq i \leq j \leq 10)$ 按列优先存入C语言的一维数组 N 中，元素 $m_{\{7,2\}}$ 在 N 中的下标是（ ）。

- A. 15
- B. 16
- C. 22
- D. 23

18. 【2021-03】 已知二维数组A按行优先方式存储，每个元素占用1个存储单元。若元素A0的存储地址是100，A3的存储地址是220，则元素A5的存储地址是（ ）。

- A. 295
- B. 300
- C. 301
- D. 306

(六) 栈、队列和数组的应用

19. 【2009-01】 为解决计算机与打印机之间速度不匹配的问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是（ ）。

- A. 栈
- B. 队列
- C. 树
- D. 图

20. 【2012-02】 已知操作符包括 $' + '、' - '、' * '、' / '、' (' 和 ') '$ 。将中缀表达式 $a+b-a*((c+d)/e-f)+g$ 转换为等价的后缀表达式 $ab+acd+e/f-* -g+$ 时，用栈来存放暂时还不能确定运算次序的操作符，若栈初始为空，则转换过程中同时保存在栈中的操作符的最大个数是（ ）。

- A. 5
- B. 7
- C. 8

D. 11

21. 【2014-02】假设栈初始为空，将中缀表达式 $a/b+(c*d-e*f)/g$ 转换为等价的后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是（ ）。

- A. $+(* -$
- B. $+(- *$
- C. $/+(* - *$
- D. $/+ - *$

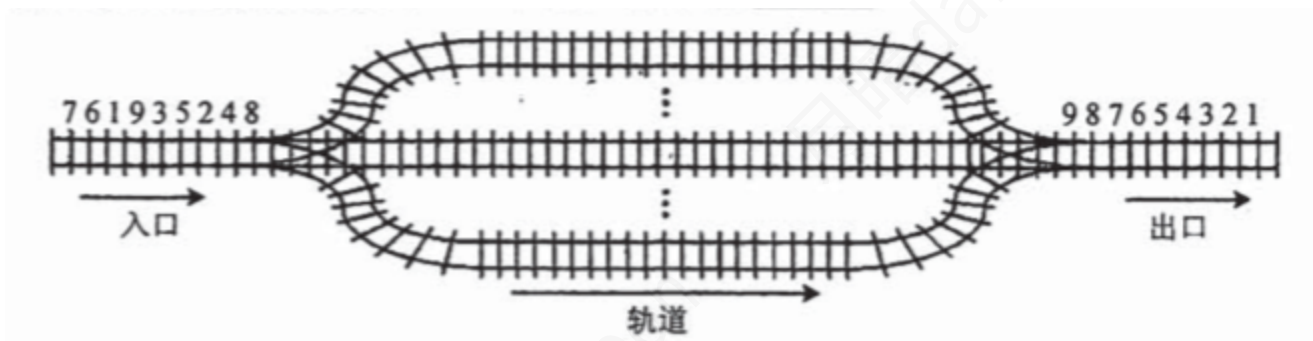
22. 【2015-01】已知程序如下：

```
int S(int n) {  
    return (n <= 0) ? 0 : S(n - 1) + n;  
}  
void main() {  
    cout << S(1);  
}
```

程序运行时使用栈来保存调用过程的信息，自栈底到栈顶保存的信息依次对应的是（ ）。

- A. $\text{main}() \rightarrow S(1) \rightarrow S(0)$
- B. $S(0) \rightarrow S(1) \rightarrow \text{main}()$
- C. $\text{main}() \rightarrow S(0) \rightarrow S(1)$
- D. $S(1) \rightarrow S(0) \rightarrow \text{main}()$

23. 【2016-04】设有下图所示的火车车轨，入口到出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为1~9的9列列车，驶入的次序依次是8, 4, 2, 5, 3, 9, 1, 6, 7。若期望驶出的次序依次为1~9，则 n 至少是（ ）。



- A. 2
- B. 3
- C. 4
- D. 5

24. 【2017-02】下列关于栈的叙述中，错误的是（ ）。

- I. 采用非递归方式重写递归程序时必须使用栈
- II. 函数调用时, 系统要用栈保存必要的信息
- III. 只要确定了入栈次序, 即可确定出栈次序
- IV. 栈是一种受限的线性表, 允许在其两端进行操作

- A. 仅 I
- B. 仅 I、II、III
- C. 仅 I、III、IV
- D. 仅 II、III、IV

25. 【2024-02】与表达式 $x+y*(z-u)/v$ 等价的后缀表达式是

- A. $xyzv-*/+$
- B. $xyzv-v/*+$
- C. $+x/*y-zuv$
- D. $+x*y/-zuv$

第四章 树与二叉树

(一) 树的基本概念

1. 【2010-05】在一棵度数为4的树 T 中, 若有20个度为4的结点, 10个度为3的结点, 1个度为2的结点, 10个度为1的结点, 则树 T 的叶结点个数是 ()。

- A. 41
- B. 82
- C. 113
- D. 122

2. 【2022-04】若三叉树T有244个结点(叶结点的高度为1), 则T的高度至少是 ()。

- A. 4
- B. 5
- C. 6
- D. 7

(二) 二叉树

二叉树的定义及其主要特征

3. 【2009-05】已知一棵完全二叉树的第6层(设根为第1层)有8个叶结点, 则完全二叉树的结点个数最多是 ()。

- A. 39
- B. 52
- C. 111
- D. 119

4. 【2011-04】若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是（ ）。
- A. 257
B. 258
C. 384
D. 385
5. 【2011-06】已知一棵有2011个结点的树，其叶结点个数为116，该树对应的二叉树中无右孩子的结点个数是（ ）。
- A. 115
B. 116
C. 1895
D. 1896
6. 【2018-04】设一棵非空完全二叉树T的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若T有 k 个叶结点，则T的结点总数是（ ）。
- A. $2k-1$
B. $2k$
C. k^2
D. 2^k-1

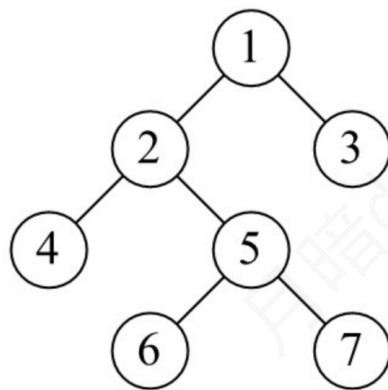
二叉树的顺序存储结构和链式存储结构

7. 【2020-03】对于任意一棵高度为5且有10个结点的二叉树，若采用顺序存储结构保存，每个结点占1个存储单元（仅存放结点的数据信息），则存放该二叉树需要的存储单元数量至少是（ ）。
- A. 31
B. 16
C. 15
D. 10
8. 【2025-03】若二叉树的节点值均为正整数，采用顺序存储方式保存在数组 R 中，用 -1 表示节点不存在，则下列数组中，不能表示一棵二叉树的是（ ）。
- A. $R[] = \{20, 15, 40, -1, -1, 35\}$
B. $R[] = \{15, 40, 10, 18, 35, -1, -1\}$
C. $R[] = \{15, 40, 10, -1, -1, -1, 12\}$
D. $R[] = \{17, 20, 35, -1, 18, 45, -1, -1, 19, 27\}$

二叉树的遍历

9. 【2009-03】给定二叉树如右图所示。设 N 代表二叉树的根， L 代表根结点的左子树， R 代表根结点的右子树。

若遍历后的结点序列为3, 1, 7, 5, 6, 2, 4, 则其遍历方式是 ()。



- A. LRN
- B. NRL
- C. RLN
- D. RNL

10. 【2011-05】若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1, 则该二叉树的中序遍历序列不会是 ()。

- A. 1, 2, 3, 4
- B. 2, 3, 4, 1
- C. 3, 2, 4, 1
- D. 4, 3, 2, 1

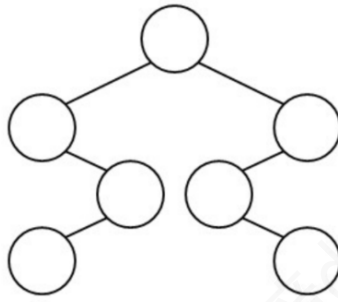
11. 【2012-03】若一棵二叉树的前序遍历序列为a, e, b, d, c, 后序遍历序列为b, c, d, e, a, 则根结点的孩子结点 ()。

- A. 只有e
- B. 有e、b
- C. 有e、c
- D. 无法确定

12. 【2015-02】先序序列为a, b, c, d的不同二叉树的个数是 ()。

- A. 13
- B. 14
- C. 15
- D. 16

13. 【2017-05】已知一棵二叉树的树形如下图所示, 其后序序列为e, a, c, b, d, g, f, 树中与结点a同层的结点是 ()。

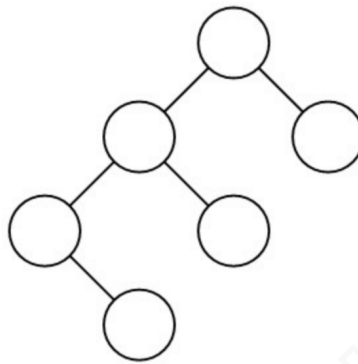


- A. c
- B. d
- C. f
- D. g

14. 【2017-04】要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的条件是（ ）。

- A. 只有左子树
- B. 只有右子树
- C. 结点的度均为1
- D. 结点的度均为2

15. 【2023-05】已知一棵二叉树的树形如下图所示，若其后序遍历为 f, d, b, e, c, a，则其先（前）序遍历序列是（ ）。



- A. a, e, d, f, b, c
- B. a, c, e, b, d, f
- C. c, a, b, e, f, d
- D. d, f, e, b, a, c

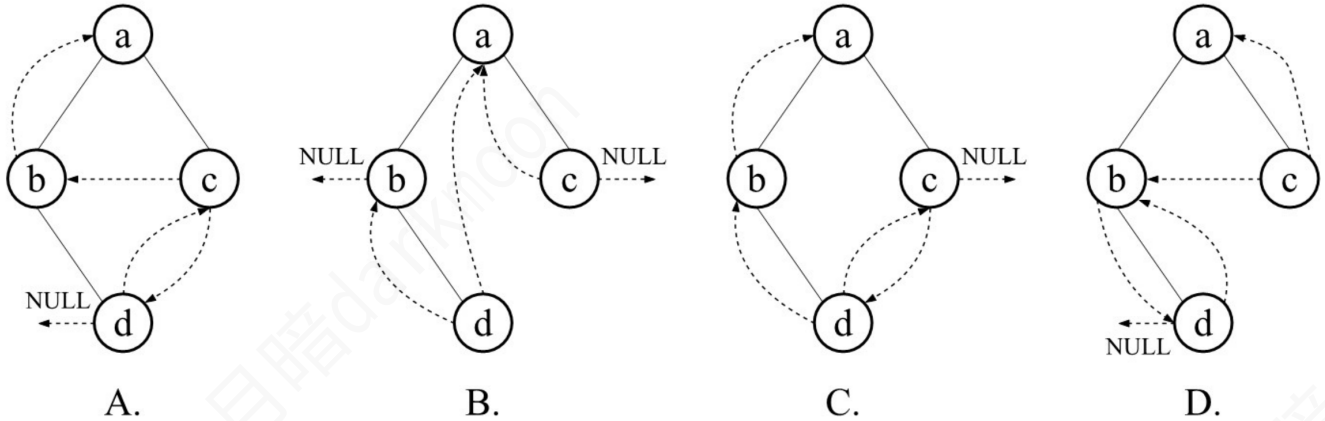
16. 【2024-03】若 p、q 和 v 均为二叉树 T 中的结点，v 有两个孩子结点，T 的中序遍历序列形如“ ...,p,v,q,... ”，则下列叙述中，正确的是

- A. p没有右孩子，q没有左孩子
- B. p没有右孩子，q有左孩子
- C. p有右孩子，q没有左孩子
- D. p有右孩子，q有左孩子

17. 【2026-03】已知二叉树 T 的中序遍历为 b, e, d, f, c, a, g。层序遍历为 a, b, g, c, d, e, f。则其后序遍历序列为多少?
- A. c, e, d, f, b, g, a
 B. c, e, f, d, b, g, a
 C. e, f, d, c, b, g, a
 D. e, g, f, d, b, c, a

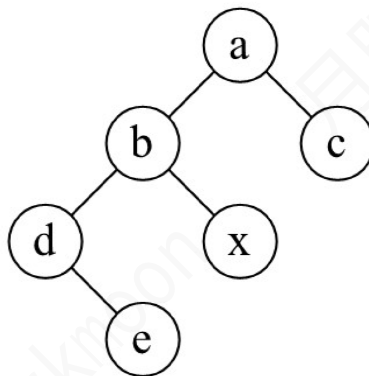
线索二叉树的基本概念和构造

18. 【2010-03】下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是（ ）



19. 【2013-05】若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y。则 X 的右线索指的是（ ）。
- A. X 的父结点
 B. 以 Y 为根的子树的最左下结点
 C. X 的左兄弟结点 Y
 D. 以 Y 为根的子树的最右下结点

20. 【2014-04】若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是（ ）。



- A. e、c
 B. e、a
 C. d、c
 D. b、a

(三) 树、森林

树的存储结构

森林与二叉树的转换

21. 【2009-06】将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 的父结点的父结点，则在原来的森林中， u 和 v 可能具有的关系是 ()。
- I. 父子关系
 - II. 兄弟关系
 - III. u 的父结点与 v 的父结点是兄弟关系
- A. 只有II
B. I和II
C. I和III
D. I、II和III
22. 【2014-05】将森林 F 转换为对应的二叉树 T ， F 中叶子的个数等于 ()。
- A. T 中叶结点的个数
B. T 中度为1的结点个数
C. T 中左孩子指针为空的结点个数
D. T 中右孩子指针为空的结点个数
23. 【2019-02】若将一棵树 T 转化为对应的二叉树 BT ，则下列对 BT 的遍历中，其遍历序列与 T 的后根遍历序列相同的是 ()。
- A. 先序遍历
B. 中序遍历
C. 后序遍历
D. 按层遍历
24. 【2020-04】已知森林 F 和对应的二叉树 T ，若 F 的先根遍历序列是 a, b, c, d, e, f ，后根遍历序列是 b, a, d, f, e, c ，则 T 的后序遍历序列是 ()。
- A. b, a, d, f, e, c
B. b, d, f, e, c, a
C. b, f, e, d, c, a
D. f, e, d, c, b, a
25. 【2021-04】某森林 F 对应的二叉树为 T ，若 T 的先序遍历序列是 a, b, d, c, e, g, f ，中序遍历序列是 b, d, a, e, g, c, f ，则 F 中树的棵数是 ()。
- A. 1
B. 2
C. 3

树和森林的遍历

(四) 树与二叉树的应用

1. 哈夫曼(Huffman)树和哈夫曼编码

26. 【2010-06】对 $n(n \geq 2)$ 个权值均不相同的字符构成哈夫曼树。下列关于该哈夫曼树的叙述中，错误的是（ ）。
- A. 该树一定是一棵完全二叉树
 - B. 树中一定没有度为 1 的结点
 - C. 树中两个权值最小的结点一定是兄弟结点
 - D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值
27. 【2013-04】已知二叉树 T 中 6 个叶结点的权分别是 2,3,4,5,6,7， T 的带权（外部）路径长度最小是（ ）。
- A. 27
 - B. 46
 - C. 54
 - D. 56
28. 【2014-06】5 个字符有如下 4 种编码方案，不是前缀编码的是（ ）。
- A. 01, 0000, 0001, 001, 1
 - B. 011, 000, 001, 010, 1
 - C. 000, 001, 010, 011, 100
 - D. 0, 100, 110, 1110, 1100
29. 【2015-03】下列选项给出的是从根分别到达两个叶结点路径上的权值序列，能属于同一棵哈夫曼树的是（ ）。
- A. 24, 10, 5 和 24, 10, 7
 - B. 24, 10, 5 和 24, 12, 7
 - C. 24, 10, 10 和 24, 14, 11
 - D. 24, 10, 5 和 24, 14, 6
30. 【2017-06】已知字符集{a, b, c, d, e, f, g, h}，若各字符的哈夫曼编码依次是 0100, 10, 0000, 0101, 001, 011, 11, 0001，则编码序列 0100011001001011110101 的译码结果是（ ）。
- A. a c g a b f h
 - B. a d b a g b b
 - C. a f b e a g d
 - D. a f e e f g d
31. 【2018-05】已知字符集{a, b, c, d, e, f}，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是（ ）。

- A. 00, 1011, 01, 1010, 11, 100
- B. 00, 100, 110, 000, 0010, 01
- C. 10, 1011, 11, 0011, 00, 010
- D. 0011, 10, 11, 0010, 01, 000

32. 【2019-03】对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有115个结点，则 n 的值是（ ）。

- A. 56
- B. 57
- C. 58
- D. 60

33. 【2021-05】若某二叉树有5个叶结点，其权值分别为10, 12, 16, 21, 30，则其最小的带权路径长度(WPL)是（ ）。

- A. 89
- B. 200
- C. 208
- D. 289

34. 【2022-05】对于任意给定的含 $n(n>2)$ 个字符集的有限集 S ，用二叉树表示 S 的哈夫曼编码集和定长编码集，分别得到二叉树 T_1 和 T_2 。下列叙述中，正确的是（ ）。

- A. T_1 和 T_2 的结点数相同
- B. T_1 的高度大于 T_2 的高度
- C. 出现频次不同的字符在 T_1 中处于不同的层
- D. 出现频次不同的字符在 T_2 中处于相同的层

35. 【2023-04】在由 6 个字符组成的字符集 S 中，各字符出现的频次分别为 3, 4, 5, 6, 8, 10，为 S 构造的哈夫曼编码的加权平均长度为（ ）。

- A. 2.4
- B. 2.5
- C. 2.67
- D. 2.75

36. 【2026-05】假设二叉树中节点权值为 $a=1, b=2, c=4, d=5, e=8, f=10, g=12$ 。当带权路径长度 (WPL) 最小时，与节点 e (权值 8) 处于相同深度的节点是哪些？

- A.d B.g C.d,f D.f,g

2. 并查集及其应用

3. 堆及其应用

第五章 图

(一)图的基本概念

1. 【2009-07】下列关于无向连通图特性的叙述中，正确的是（ ）。
 - I. 所有顶点的度之和为偶数
 - II. 边数大于顶点个数减1
 - III. 至少有一个顶点的度为1

A. 只有I
B. 只有II
C. I和II
D. I和III
2. 【2010-07】若无向图 $G=(V,E)$ 中含有 7 个顶点，要保证图 G 在任何情况下都是连通的，则需要的边数最少是（ ）。

A. 6 B. 15 C. 16 D. 21
3. 【2017-07】已知无向图G含有16条边，其中度为4的顶点个数为3，度为3的顶点个数为4，其他顶点的度均小于3。图G所含的顶点个数至少是（ ）。

A. 10
B. 11
C. 13
D. 15
4. 【2022-06】对于无向图 $G=(V,E)$ ，下列选项中正确的是（ ）。

A. 当 $|V|>|E|$ 时，G 一定是连通的
B. 当 $|V|<|E|$ ，G 一定是连通的
C. 当 $|V|=|E|-1$ ，G 一定是不连通的
D. 当 $|V|>|E|+1$ ，G 一定是不连通的

(二)图的存储及基本操作

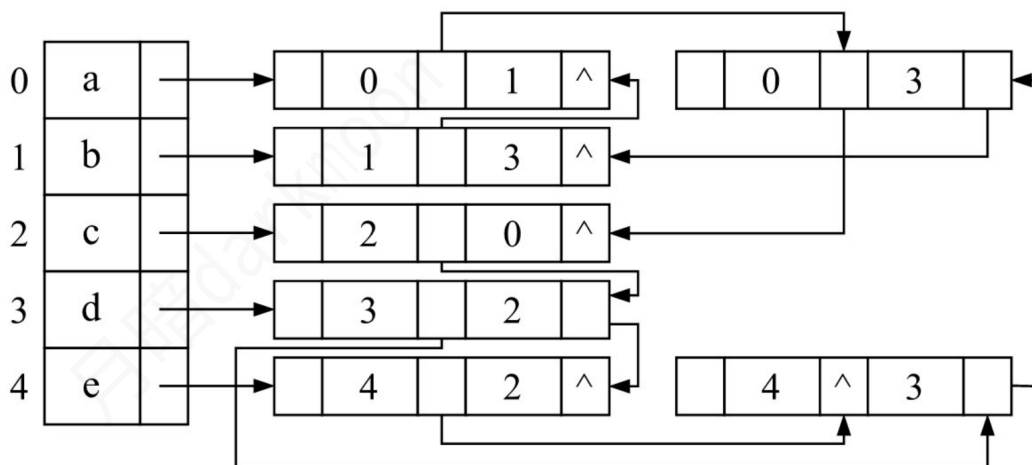
1. 邻接矩阵
2. 邻接表
3. 邻接多重表、十字链表

5. 【2013-07】设图的邻接矩阵 A 如下所示。各顶点的度依次是（ ）。

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

- A. 1, 2, 1, 2
- B. 2, 2, 1, 1
- C. 3, 4, 2, 3
- D. 4, 4, 2, 2

6. 【2024-04】若无向图 $G=(V,E)$ 的邻接多重表如下图所示，则 G 中顶点 b 与 d 的度分别是



- A. 0, 2
- B. 2, 4
- C. 2, 5
- D. 3, 4

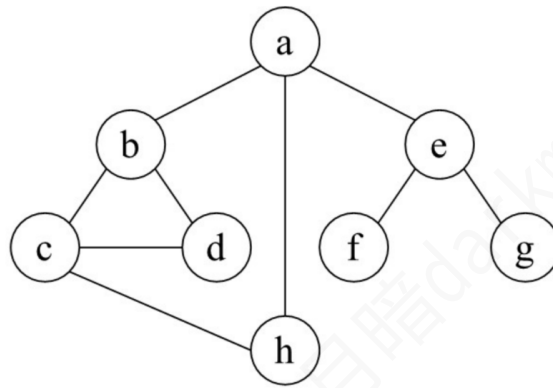
(三)图的遍历

1. 深度优先搜索
2. 广度优先搜索

7. 【2012-05】对有 n 个结点、 e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法时间复杂度是 ()。

- A. $O(n)$
- B. $O(e)$
- C. $O(n+e)$
- D. $O(ne)$

8. 【2013-08】若对如下无向图进行遍历，则下列选项中，不是广度优先遍历序列的是 ()。

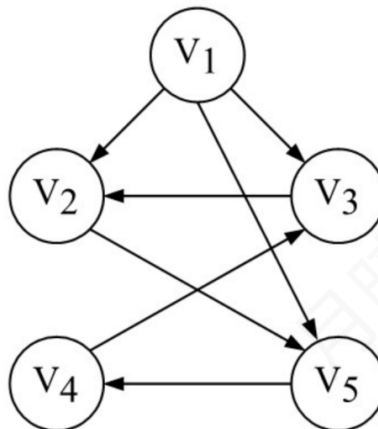


- A. h, c, a, b, d, e, g, f
- B. e, a, f, g, b, h, c, d
- C. d, b, c, a, h, e, f, g
- D. a, b, c, d, h, e, f, g

9. 【2015-05】 设有向图 $G=(V,E)$ ，顶点集 $V=\{V_0,V_1,V_2,V_3\}$ ，边集 $E=\{(V_0,V_1),(V_0,V_2),(V_0,V_3),(V_1,V_3)\}$ 。若从顶点 V_0 开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是（ ）。

- A. 2
- B. 3
- C. 4
- D. 5

10. 【2016-06】 下列选项中，不是下图深度优先搜索序列的是（ ）。



- A. V_1, V_5, V_4, V_3, V_2
- B. V_1, V_3, V_2, V_5, V_4
- C. V_1, V_2, V_5, V_4, V_3
- D. V_1, V_2, V_3, V_4, V_5

(四)图的基本应用

11. 【2019-06】 用有向无环图描述表达式 $(x+y)((x+y)/x)$ ，需要的顶点个数至少是（ ）。

- A. 5

- B. 6
- C. 8
- D. 9

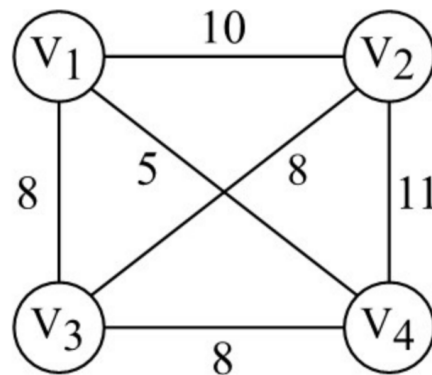
最小(代价)生成树

12. 【2012-08】 下列关于最小生成树的叙述中，正确的是（ ）。

- I. 最小生成树的代价唯一
- II. 所有权值最小的边一定会出现在所有的最小生成树中
- III. 使用普里姆（Prim）算法从不同顶点开始得到的最小生成树一定相同
- IV. 使用普里姆算法和克鲁斯卡尔（Kruskal）算法得到的最小生成树总不同

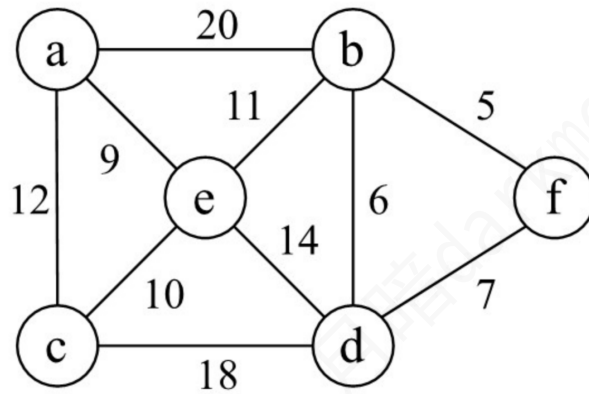
- A. 仅I
- B. 仅II
- C. 仅I、III
- D. 仅II、IV

13. 【2015-06】 求下面带权图的最小（代价）生成树时，可能是克鲁斯卡尔（Kruskal）算法第2次选中但不是普里姆（Prim）算法（从 V_4 开始）第2次选中的边是（ ）。



- A. (V_1, V_3)
- B. (V_1, V_4)
- C. (V_2, V_3)
- D. (V_3, V_4)

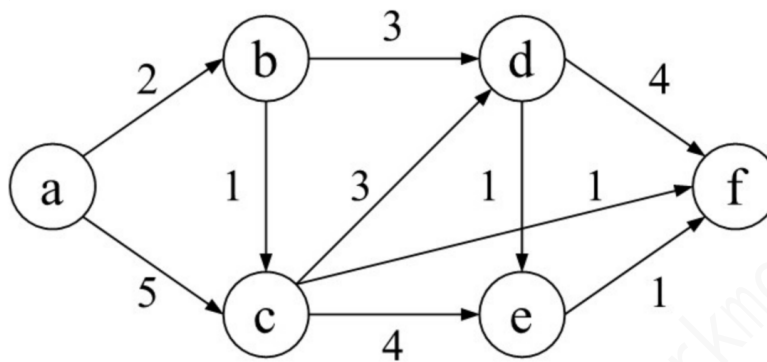
14. 【2020-07】 已知无向图G如下图所示，使用克鲁斯卡尔（Kruskal）算法求图G的最小生成树，加到最小生成树中的边依次是（ ）。



- A. (b, f), (b, d), (a, e), (c, e), (b, e)
- B. (b, f), (b, d), (b, e), (c, e), (a, e)
- C. (a, e), (b, e), (c, e), (b, d), (b, f)
- D. (a, e), (c, e), (b, e), (b, f), (b, d)

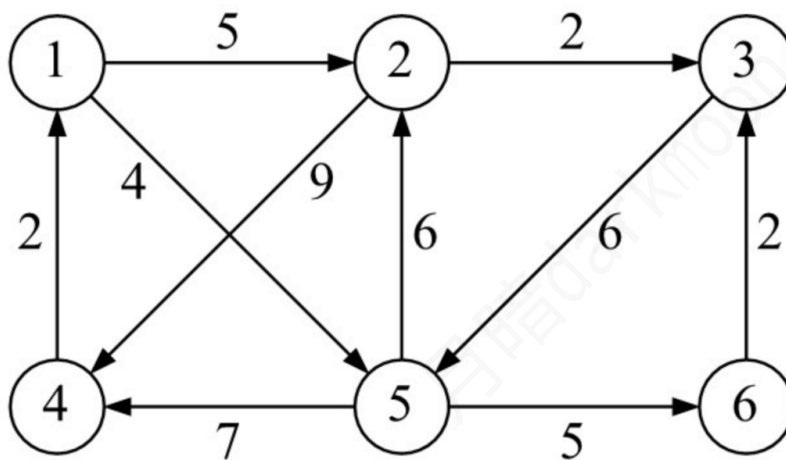
最短路径

15. 【2012-07】对如下有向图带权图，若采用迪杰斯特拉（Dijkstra）算法求从源点a到其他各顶点的最短路径，则得到的第一条最短路径的目标顶点是b，第二条最短路径的目标顶点是c，后续得到的其余最短路径的目标顶点依次是（ ）。



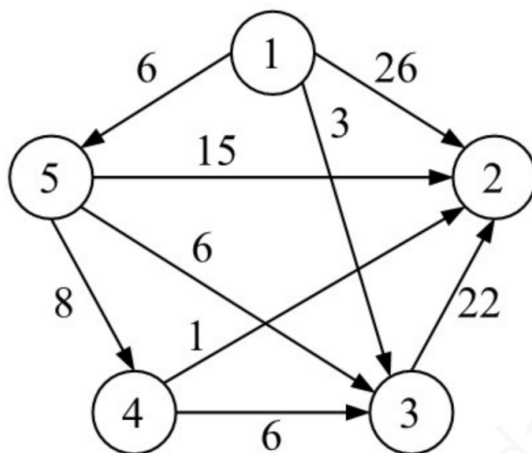
- A. d, e, f
- B. e, d, f
- C. f, d, e
- D. f, e, d

16. 【2016-08】使用迪杰斯特拉（Dijkstra）算法求下图中从顶点 1 到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是（ ）。



- A. 5, 2, 3, 4, 6
- B. 5, 2, 3, 6, 4
- C. 5, 2, 4, 3, 6
- D. 5, 2, 6, 3, 4

17. 【2021-08】使用Dijkstra算法求下图中从顶点1到其余各顶点的最短路径，将当前找到的从顶点1到顶点2, 3, 4, 5的最短路径长度保存在数组dist中，求出第二条最短路径后，dist中的内容更新为（ ）。



- A. 26, 3, 14, 6
- B. 25, 3, 14, 6
- C. 21, 3, 14, 6
- D. 15, 3, 14, 6

18. 【2023-06】已知无向连通图 G 中各边的权值均为 1，下列算法中，一定能够求出图 G 中从某顶点到其余各顶点最短路径的是（ ）。

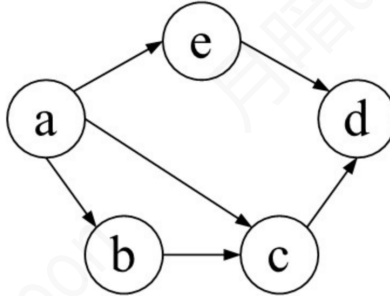
- I. 普利姆 (Prim) 算法
- II. 克鲁斯卡尔 (Kruskal) 算法
- III. 图的广度优先搜索算法

- A. 仅 I
- B. 仅 III

- C. 仅 I 和 II
- D. I、II 和 III

拓扑排序

19. 【2010-08】对下图进行拓扑排序，可以得到不同拓扑序列的个数是（ ）。



- A. 4
 - B. 3
 - C. 2
 - D. 1
20. 【2011-08】下列关于图的叙述中，正确的是（ ）。

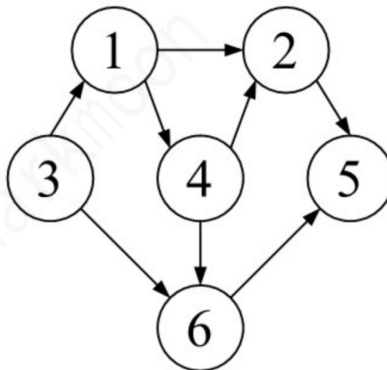
- I. 回路是简单路径
- II. 存储稀疏图，用邻接矩阵比邻接表更省空间
- III. 若有向图中存在拓扑序列，则该图不存在回路

- A. 仅 II
- B. 仅 I、II
- C. 仅 III
- D. 仅 I、III

21. 【2012-06】若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑序列的结论是（ ）。

- A. 存在，且唯一
- B. 存在，且不唯一
- C. 存在，可能不唯一
- D. 无法确定是否存在

22. 【2014-07】对如下所示的有向图进行拓扑排序，得到的拓扑序列可能是（ ）。

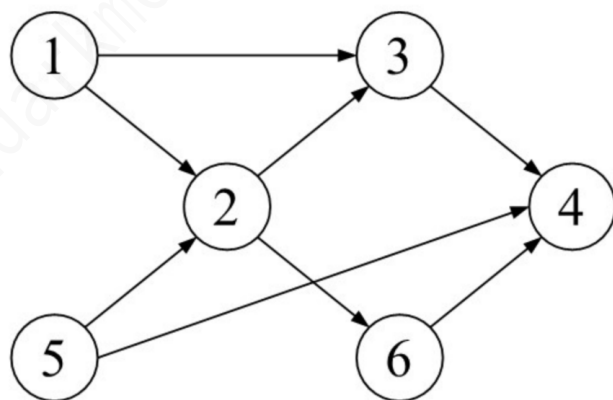


- A. 3, 1, 2, 4, 5, 6
- B. 3, 1, 2, 4, 6, 5
- C. 3, 1, 4, 2, 5, 6
- D. 3, 1, 4, 2, 6, 5

23. 【2016-07】 若将 n 个顶点 e 条弧的有向图采用邻接表存储，则拓扑排序算法的时间复杂度是（ ）。

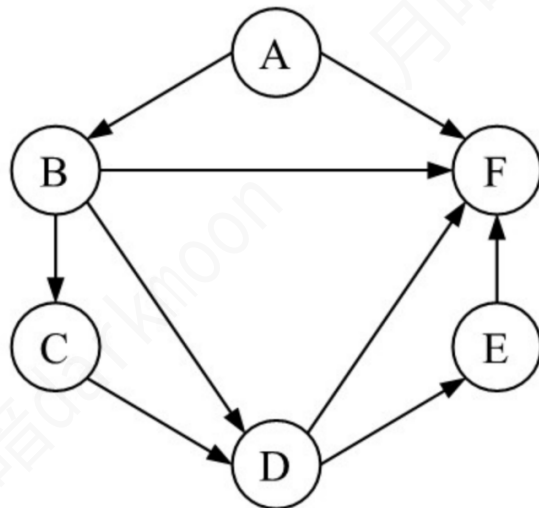
- A. $O(n)$
- B. $O(n+e)$
- C. $O(n^2)$
- D. $O(ne)$

24. 【2018-07】 下列选项中，不是如下有向图的拓扑序列的是（ ）。



- A. 1, 5, 2, 3, 6, 4
- B. 5, 1, 2, 6, 3, 4
- C. 5, 1, 2, 3, 6, 4
- D. 5, 2, 1, 6, 3, 4

25. 【2021-07】 给定如下有向图，该图的拓扑有序序列的个数（ ）。



- A. 1

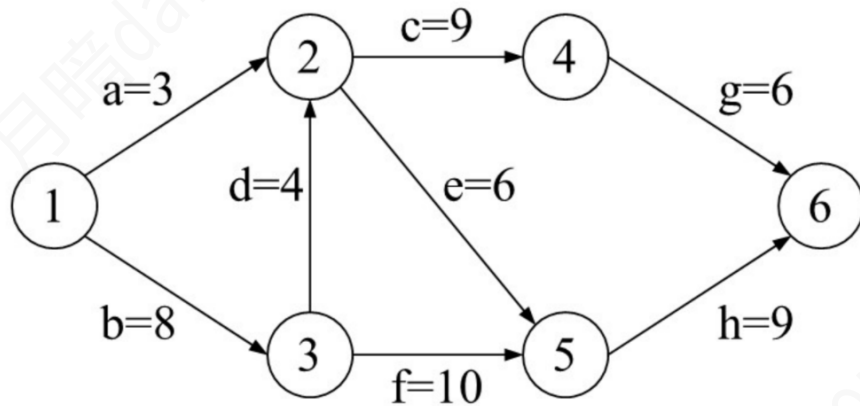
- B. 2
- C. 3
- D. 4

26. 【2020-06】修改递归方式实现图的深度优先搜索（DFS）算法，将输出（访问）顶点信息的语句移到退出递归前（即执行输出语句后立刻退出递归）。采用修改后的算法遍历有向无环图G，若输出结果中包含G中全部顶点，则输出的顶点序列是G的（ ）。

- A. 拓扑有序序列
- B. 逆拓扑有序序列
- C. 广度优先搜索序列
- D. 深度优先搜索序列

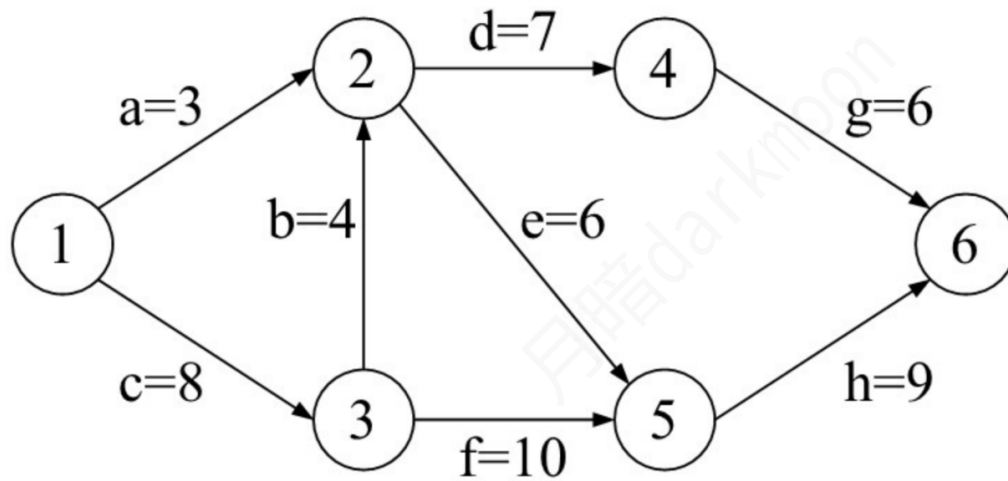
关键路径

27. 【2013-09】下列AOE网表示一项包含8个活动的工程，通过同时加快若干活动的进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（ ）。



- A. c 和 e
- B. d 和 c
- C. f 和 d
- D. f 和 h

28. 【2019-05】下图所示的 AOE 网表示一项包含 8 个活动的工程。活动 d 的最早开始时间和最迟开始时间分别是（ ）。

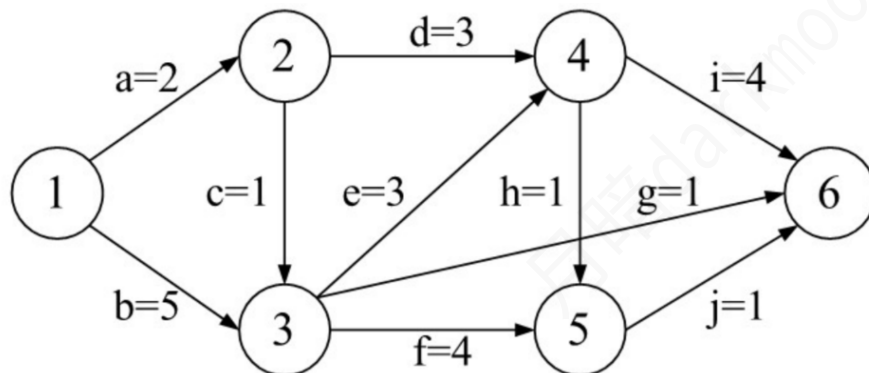


- A. 3和7
- B. 12和12
- C. 12和14
- D. 15和15

29. 【2020-08】若采用AOE网估算工程进度，则下列叙述中正确的是（ ）。

- A. 关键路径是从源点到汇点边数最多的一条路径
- B. 关键路径是从源点到汇点路径长度最长的路径
- C. 增加任一关键活动的时间不会延长工程的工期
- D. 缩短任一关键活动的时间将会缩短工程的工期

30. 【2022-07】下图是有 10 个活动的AOE网，时间余量最大的活动是（ ）。



- A. c
- B. g
- C. h
- D. j

第六章 查找

(一)查找的基本概念

(二)顺序查找法

(三)分块查找法

(四)折半查找法

1. 【2010-09】已知一个长度为 16 的顺序表 L，其元素按关键字有序排列。若采用折半查找法查找一个 L 中不存在的元素，则关键字的比较次数最多是（ ）。

- A. 4
- B. 5
- C. 6
- D. 7

2. 【2015】下列选项中不能构成折半查找中关键字比较序列的是()。

- A. 500, 200, 450, 180
- B. 500, 450, 200, 180
- C. 180, 500, 200, 450
- D. 180, 200, 500, 450

3. 【2015-07】下列选项中，不能构成折半查找中关键字比较序列的是（ ）。

- A. 500, 200, 450, 180
- B. 500, 450, 200, 180
- C. 180, 500, 200, 450
- D. 180, 200, 500, 450

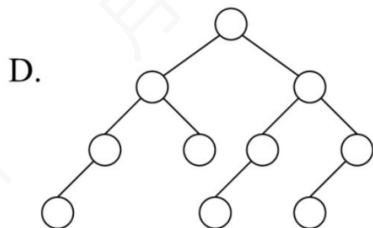
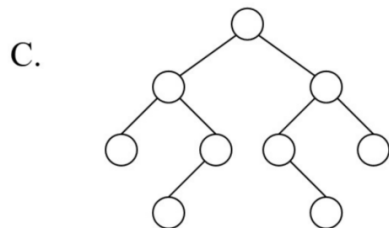
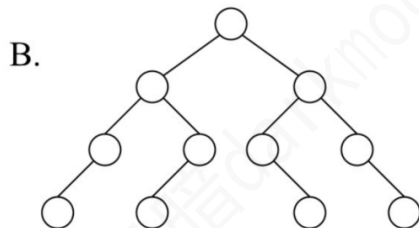
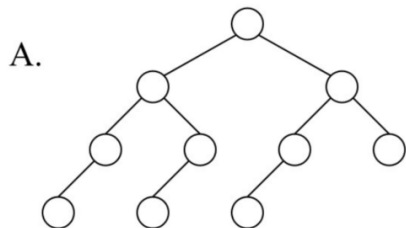
4. 【2016-09】在有 $n(n > 1000)$ 个元素的升序数组 A 中查找关键字 x。查找算法的伪代码如下所示。

```
1  k = 0;
2  while (k < n 且 A[k] < x) k = k + 3;
3  if (k < n 且 A[k] == x) 查找成功;
4  else if (k - 1 < n 且 A[k - 1] == x) 查找成功;
5  else if (k - 2 < n 且 A[k - 2] == x) 查找成功;
6  else 查找失败;
```

本算法与折半查找算法相比，有可能具有更少比较次数的情形是（ ）。

- A. 当 x 不在数组中
- B. 当 x 接近数组开头处
- C. 当 x 接近数组结尾处
- D. 当 x 位于数组中间位置

5. 【2017-08】 下列二叉树中，可能成为折半查找判定树（不含外部结点）的是（ ）。



6. 【2023-08】 对含 600 个元素的有序顺序表进行折半查找，关键字间的比较次数最多是（ ）。

- A. 9
- B. 10
- C. 30
- D. 300

7. 【2024-05】 下列数据结构中，不适合直接使用折半查找的是（ ）。

- I. 有序链表
- II. 无序数组
- III. 有序静态链表
- IV. 无序静态链表

- A. 仅I, III
- B. 仅II, IV
- C. 仅II, III, IV
- D. I, II, III, IV

(五)树型查找

二叉搜索树

8. 【2011-07】 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是（ ）。

- A. 95, 22, 91, 24, 94, 71
- C. 21, 89, 77, 29, 36, 38
- B. 92, 20, 91, 34, 88, 35
- D. 12, 25, 71, 68, 33, 34

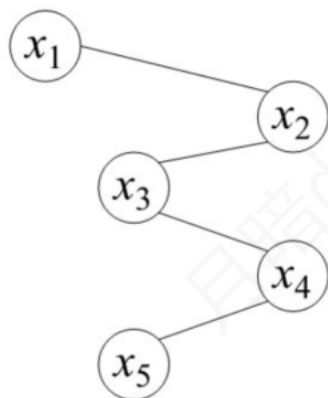
9. 【2013-06】 在任意一棵非空二叉排序树 T1 中，删除某结点 v 之后形成二叉排序树 T2，再将 v 插入 T2 形成二

叉排序树 T_3 。下列关于 T_1 与 T_3 的叙述中，正确的是（ ）。

- I. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 不同
- II. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 相同
- III. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 不同
- IV. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 相同

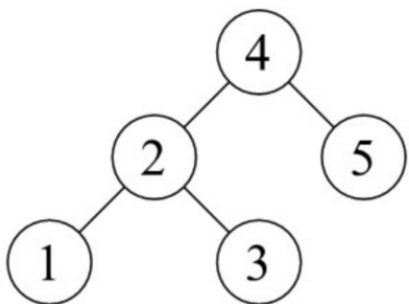
- A. 仅I、III
- B. 仅I、IV
- C. 仅II、III
- D. 仅II、IV

10. 【2018-06】已知二叉排序树如下图所示，元素之间应满足的大小关系是（ ）。



- A. $x_1 < x_2 < x_3$
- B. $x_1 < x_4 < x_5$
- C. $x_3 < x_5 < x_4$
- D. $x_4 < x_3 < x_5$

11. 【2020-05】下列给定的关键字输入序列中，不能生成如下二叉排序树的是（ ）。



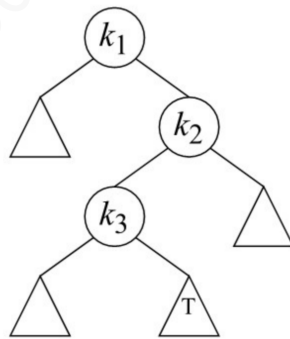
- A. 4, 5, 2, 1, 3
- B. 4, 5, 1, 2, 3
- C. 4, 2, 5, 3, 1
- D. 4, 2, 1, 3, 5

12. 【2022-03】若结点 p 与 q 在二叉搜索树 T 中的中序遍历中相邻，且 p 在 q 之前，则下列 p 与 q 的关系中，不可能的是

()。

- I. q是p的双亲
 - II. q是p的右孩子
 - III. q是p的右兄弟
 - IV. q是p的双亲的双亲
- A. 仅I
B. 仅III
C. 仅II、III
D. 仅II、IV

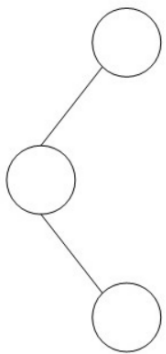
13. 【2024-07】一棵二叉搜索树如题 7 图所示， k_1 、 k_2 、 k_3 分别是对应结点中保存的关键字。子树 T 的任一结点中保存的关键字 x 满足的是



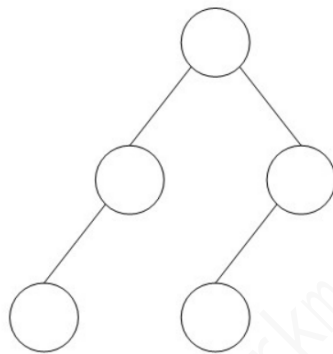
- A. $x < k_1$
B. $x > k_2$
C. $k_1 < x < k_3$
D. $k_3 < x < k_2$

平衡二叉树

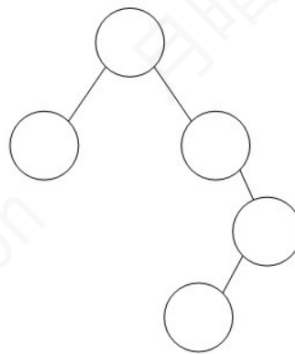
14. 【2009-04】下列二叉排序树中，满足平衡二叉树定义的是 ()。



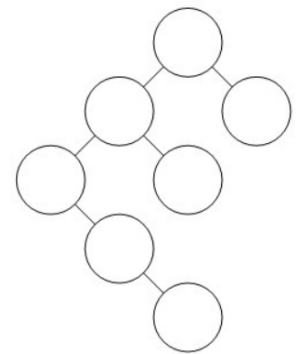
A.



B.

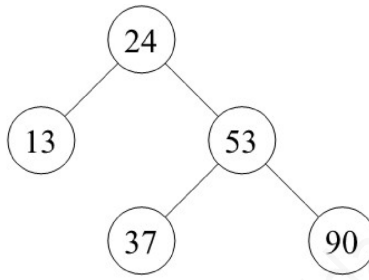


C.



D.

15. 【2010-04】在下图所示的平衡二叉树中，插入关键字48后得到一棵新平衡二叉树。在新平衡二叉树中，关键字37所在结点的左、右子结点中保存的关键字分别是 ()。



- A. 13, 48
- B. 24, 48
- C. 24, 53
- D. 24, 90

16. 【2012-04】若平衡二叉树平的高度为 6，且所有非叶结点的平衡因子均为 1，则该平衡二叉树的结点总数为（ ）。

- A. 10
- B. 20
- C. 32
- D. 33

17. 【2013-03】若将关键字 1,2,3,4,5,6,7 依次插入到初始为空的平衡二叉树 T 中，则 T 中平衡因子为 0 的分支结点的个数是（ ）。

- A. 0
- B. 1
- C. 2
- D. 3

18. 【2015-04】现有一棵无重复关键字的平衡二叉树（AVL树），对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是（ ）。

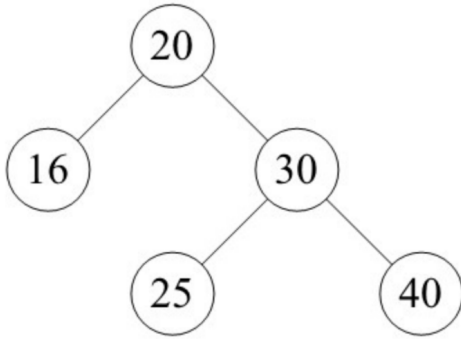
- A. 根结点的度一定为2
- B. 树中最小元素一定是叶结点
- C. 最后插入的元素一定是叶结点
- D. 树中最大元素一定是无左子树

19. 【2019-04】在任意一棵非空平衡二叉树（AVL树）T1 中，删除某结点 v 之后形成平衡二叉树 T2，再将 v 插入 T2 形成平衡二叉树 T3。下列关于 T1 与 T3 的叙述中，正确的是（ ）。

- I.若 v 是 T1 的叶结点，则 T1 与 T3 可能不相同
 - II.若 v 不是 T1 的叶结点，则 T1 与 T3 一定不相同
 - III.若 v 不是 T1 的叶结点，则 T1 与 T3 一定相同
- A. 仅I
 - B. 仅II
 - C. 仅I、II

D. 仅I、III

20. 【2021-06】 给定平衡二叉树如下图所示，插入关键字23后，根中的关键字是（ ）。



- A. 16
- B. 20
- C. 23
- D. 25

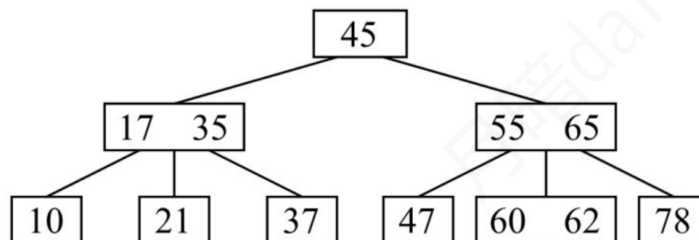
红黑树

(六)B树及其基本操作、B+树的基本概念

21. 【2009-08】 下列叙述中，不符合 m 阶B树定义要求的是（ ）。

- A. 根结点最多有 m 棵子树
- B. 所有叶结点都在同一层上
- C. 各结点内关键字均升序或降序排列
- D. 叶结点之间通过指针链接

22. 【2012-09】 已知一棵3阶B树，如下图所示。删除关键字78得到一棵新B树，其最右叶结点中的关键字是（ ）。



- A. 60
- B. 60, 62
- C. 62, 65
- D. 65

23. 【2013-10】 在一棵高度为2的5阶B树中，所含关键字的个数最少是（ ）。

- A. 5
- B. 7

C. 8

D. 14

24. 【2014-09】在一颗具有15个关键字的4阶B树中，含关键字的结点个数最多的是（ ）。

A. 5

B. 6

C. 10

D. 15

25. 【2016-10】B+树不同于B树的特点之一是（ ）。

A. 能支持顺序查找

B. 结点中含有关键字

C. 根结点至少有两个分支

D. 所有叶结点都在同一层上

26. 【2017-09】下列应用中，适合使用 B+ 树的是（ ）。

A. 编译器中的词法分析

B. 关系数据库系统中的索引

C. 网络中的路由表快速查找

D. 操作系统的磁盘空闲块管理

27. 【2018-08】高度为5的3阶B树含有的关键字个数至少是（ ）。

A. 15

B. 31

C. 62

D. 242

28. 【2020-10】依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是（ ）。

A. 8

B. 6, 9

C. 8, 13

D. 9, 12

29. 【2021-09】在一棵高度为3的3阶B树中，根为第1层，若第2层中有4个关键字，则该树的结点个数最多（ ）。

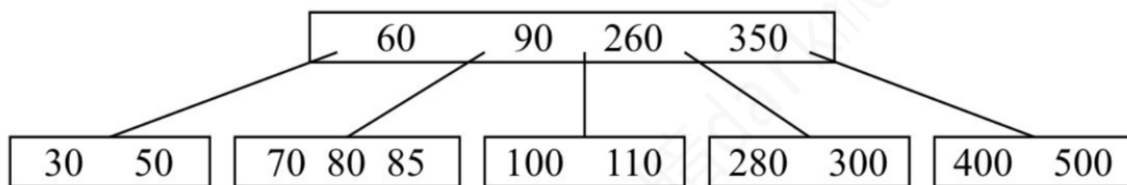
A. 11

B. 10

C. 9

D. 8

30. 【2022-08】在下图所示的5阶B树 T 中，删除关键字 260 之后需要进行必要的调整，得到新的B树 T1。下列选项中，不可能是 T1 根结点中的关键字序列的是（ ）。



- A. 60,90,280
- B. 60,90,350
- C. 60,85,110,350
- D. 60,90,110,350

31. 【2023-07】下列关于非空 B 树的叙述中，正确的是（ ）。

- I. 插入操作可能增加树的高度
- II. 删除操作一定会导致叶结点的变化
- III. 查找某关键字总是要查找到叶结点
- IV. 插入的新关键字最终位于叶结点中

- A. 仅 I
- B. 仅 I 和 II
- C. 仅 III 和 IV
- D. 仅 I、II 和 IV

(七)散列(Hash)表

32. 【2011-09】为提高散列 (Hash) 表的查找效率，可以采取的正确措施是（ ）。

- I. 增大装填 (载) 因子
- II. 设计冲突 (碰撞) 少的散列函数
- III. 处理冲突 (碰撞) 时避免产生聚集 (堆积) 现象

- A. 仅 I
- B. 仅 II
- C. 仅 I、II
- D. 仅 II、III

33. 【2014-08】用哈希 (散列) 方法处理冲突 (碰撞) 时可能出现堆积 (聚焦) 现象，下列选项中，会受堆积现象直接影响的是（ ）。

- A. 存储效率
- B. 散列函数
- C. 装填 (装载) 因子
- D. 平均查找长度

34. 【2018-09】 现有长度为7、初始为空的散列表HT，散列函数 $H(k) = k \% 7$ ，用线性探测再散列法解决冲突。将关键字22, 43, 15依次插入HT，查找成功的平均查找长度是（ ）。

- A. 1.5
- B. 1.6
- C. 2
- D. 3

35. 【2019-08】 现有长度为11且初始为空的散列表HT，散列函数是 $H(key) = key \% 7$ ，采用线性探查（线性探测再散列）法解决冲突。将关键字序列87, 40, 30, 6, 11, 22, 98, 20依次插入HT后，HT查找失败的平均查找长度是（ ）。

- A. 4
- B. 5.25
- C. 6
- D. 6.29

36. 【2022-09】 影响散列（哈希）方法平均查找长度的是（ ）。

- I. 装填因子
- II. 散列函数
- III. 冲突解决策略

- A. 仅I、II
- B. 仅I、III
- C. 仅II、III
- D. I、II、III

37. 【2023-09】 现有长度为 5，初始为空的散列表 HT，散列函数 $H(k)=(k+4) \bmod 5$ ，用线性探查再散列法解决冲突。若将关键字序列 2022, 12, 25 依次插入 HT 中，然后删除关键字 25，则 HT 中查找失败的平均查找长度为（ ）。

- A. 1
- B. 1.6
- C. 1.8
- D. 2.2

(八)字符串模式匹配

38. 【2015-08】 已知字符串S为“abaabaabacacaabaabcc”，模式串t为“abaabc”。采用KMP算法进行匹配，第一次出现“失配”($S[i] \neq t[j]$)时， $i = j = 5$ ，则下次开始匹配时， i 和 j 的值分别是（ ）。

- A. $i = 1, j = 0$
- B. $i = 5, j = 0$
- C. $i = 5, j = 2$
- D. $i = 6, j = 2$

39. 【2019-09】设主串 $T = \text{"abaabaabcabaabc"}$ ，模式串 $S = \text{"abaabc"}$ ，采用 KMP 算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是（ ）。

- A. 9
- B. 10
- C. 12
- D. 15

40. 【2024-06】KMP 算法使用修正后的 next 数组进行模式匹配，模式串 $S = \text{"aabaab"}$ ，当主串中某字符与 S 中某字符失配时， S 向右滑动的最长距离是

- A. 5
- B. 4
- C. 3
- D. 2

(九)查找算法的分析及应用

第七章 排序

(一)排序的基本概念

(二)直接插入排序

1. 【2022-11】对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是（ ）。

- I. 大部分元素已有序
- II. 待排序元素数量很少
- III. 要求空间复杂度为 $O(1)$
- IV. 要求排序算法是稳定的

- A. 仅I、II
- B. 仅III、IV
- C. 仅I、II、IV
- D. I、II、III、IV

(三)折半插入排序

2. 【2012-11】对一待排序序列分别进行折半插入排序和直接插入排序，两者之间可能的不同之处是（ ）。

- A. 排序的总趟数
- B. 元素的移动次数
- C. 使用辅助空间的数量
- D. 元素之间的比较次数

(四)起泡排序(bubble sort)

(五)简单选择排序

(六)希尔排序(shell sort)

3. 【2014-10】用希尔排序方法对一个数据序列进行排序时，若第1趟排序结果为9, 1, 4, 13, 7, 8, 20, 23, 15，则该趟排序采用的增量（间隔）可能是（ ）。
- A. 2
B. 3
C. 4
D. 5
4. 【2015-11】希尔排序的组内排序采用的是（ ）。
- A. 直接插入排序
B. 折半插入排序
C. 快速排序
D. 归并排序
5. 【2018-10】对初始数据序列(8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6)进行希尔排序。若第一趟排序结果为(1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8)，第二趟排序结果为(1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9)，则两趟排序采用的增量（间隔）依次是（ ）。
- A. 3, 1
B. 3, 2
C. 5, 2
D. 5, 3

(七)快速排序

6. 【2010-10】采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中，正确的是（ ）。
- A. 递归次数与初始数据的排列次序无关
B. 每次划分后，先处理较长的分区可以减少递归次数
C. 每次划分后，先处理较短的分区可以减少递归次数
D. 递归次数与每次划分后得到的分区的处理顺序无关
7. 【2011-10】为实现快速排序算法，待排序序列宜采用的存储方式是（ ）。
- A. 顺序存储
B. 散列存储
C. 链式存储
D. 索引存储

8. 【2014-11】下列选项中，不可能是快速排序第2趟排序结果的是（ ）。

- A. 2, 3, 5, 4, 6, 7, 9
- B. 2, 7, 5, 6, 4, 3, 9
- C. 3, 2, 5, 4, 7, 6, 9
- D. 4, 2, 3, 5, 7, 6, 9

9. 【2019-10】排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中，不可能是快速排序第二趟结果的是（ ）。

- A. 5, 2, 16, 12, 28, 60, 32, 72
- B. 2, 16, 5, 28, 12, 60, 32, 72
- C. 2, 12, 16, 5, 28, 32, 72, 60
- D. 5, 2, 12, 28, 16, 32, 72, 60

10. 【2022-11】对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是（ ）。

- I. 大部分元素已有序
 - II. 待排序元素数量很少
 - III. 要求空间复杂度为 $O(1)$
 - IV. 要求排序算法是稳定的
- A. 仅I、II
 - B. 仅III、IV
 - C. 仅I、II、IV
 - D. I、II、III、IV

11. 【2023-11】使用快速排序算法对数据进行升序排序，若经过一次划分后得到的数据序列是 68, 11, 70, 23, 80, 77, 48, 81, 93, 88，则该次划分的轴枢是（ ）。

- A. 11
- B. 70
- C. 80
- D. 81

12. 【2024-08】使用快速排序对含 $n(n \geq 3)$ 个元素的数组M进行排序，若第一趟排序将除M中轴枢外的 $n-1$ 个元素划分为均不为空的 P 和 Q 两个块，则下列叙述中，正确的是

- A. P 和 Q 块间有序
- B. P 和 Q 均块内有序
- C. P 和 Q 的元素个数大致相等
- D. P 和 Q 中均不存在相等的元素

(八)堆排序

13. 【2009-09】已知关键字序列5, 8, 12, 19, 28, 20, 15, 22是小根堆（最小堆），插入关键字3，调整后得到的小根堆是（ ）。

- A. 3, 5, 12, 8, 28, 20, 15, 22, 19
- B. 3, 5, 12, 19, 20, 15, 22, 8, 28
- C. 3, 8, 12, 5, 20, 15, 22, 28, 19
- D. 3, 12, 5, 8, 28, 20, 15, 22, 19

14. 【2011-11】已知序列25, 13, 10, 12, 9是大根堆，在序列尾部插入新元素18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是（ ）。

- A. 1
- B. 2
- C. 4
- D. 5

15. 【2015-10】已知小根堆为8, 15, 10, 21, 34, 16, 12，删除关键字8之后需重建堆，在此过程中，关键字之间的比较次数是（ ）。

- A. 1
- B. 2
- C. 3
- D. 4

16. 【2018-11】将数据序列〈6,1,5,9,8,4,7〉建成大根堆时，正确的序列变化过程

- A. 〈6,1,7,9,8,4,5〉→〈6,9,7,1,8,4,5〉→〈9,6,7,1,8,4,5〉→〈9,8,7,1,6,4,5〉
- B. 〈6,9,5,1,8,4,7〉→〈6,9,7,1,8,4,5〉→〈9,6,7,1,8,4,5〉→〈9,8,7,1,6,4,5〉
- C. 〈6,9,5,1,8,4,7〉→〈9,6,5,1,8,4,7〉→〈9,6,7,1,8,4,5〉→〈9,8,7,1,6,4,5〉
- D. 〈6,1,7,9,8,4,5〉→〈7,1,6,9,8,4,5〉→〈7,9,6,1,8,4,5〉→〈9,7,6,1,8,4,5〉→〈9,8,7,1,6,4,5〉

17. 【2020-09】下列关于大根堆（至少含2个元素）的叙述中，正确的是（ ）。

- I. 可以将堆看成是一棵完全二叉树
 - II. 可以采用顺序存储方式保存堆
 - III. 可以将堆看成是一棵二叉排序树
 - IV. 堆中的次大值一定在根的下一层
- A. 仅I、II
 - B. 仅II、III
 - C. 仅I、II和IV
 - D. 仅I、III和IV

18. 【2021-11】将关键字6, 9, 1, 5, 8, 4, 7依次插入到初始为空的大根堆H中，得到的H是（ ）。

- A. 9, 8, 7, 6, 5, 4, 1
- B. 9, 8, 7, 5, 6, 1, 4
- C. 9, 8, 7, 5, 6, 4, 1

D. 9, 6, 7, 5, 8, 4, 1

19. 【2024-09】已知关键字序列 28, 22, 20, 19, 8, 12, 15, 5 是大根堆（最大堆），对该堆进行两次删除操作后，得到的新堆是

- A. 20, 19, 15, 12, 8, 5
- B. 20, 19, 15, 5, 8, 12
- C. 20, 19, 12, 15, 8, 5
- D. 20, 19, 8, 12, 15, 5

(九)二路归并排序(merge sort)

20. 【2017-10】在内部排序时，若选择了归并排序而没有选择插入排序，则可能的理由是（ ）。

- I. 归并排序的程序代码更短
- II. 归并排序的占用空间更少
- III. 归并排序的运行效率更高

- A. 仅 II
- B. 仅 III
- C. 仅 I、II
- D. 仅 I、III

21. 【2022-10】使用二路归并排序对含 n 个元素的数组 M 进行排序时，二路归并排序操作的功能是（ ）。

- A. 将两个有序表合并为一个新的有序表
- B. 将 M 划分为两部分，两部分的元素个数大致相等
- C. 将 M 分成 n 个部分，每个部分中仅含有一个元素
- D. 将 M 划分为两部分，一部分元素的值均小于另一部分元素值

22. 【2024-10】现有由关键字组成的3个有序序列 (3,5), (7,9) 和 (6)，若按从左至右的次序选择有序序列进行二路归并排序，则关键字之间的总比较次数是

- A. 3
- B. 4
- C. 5
- D. 6

(十)基数排序

23. 【2013-11】对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序，则第2趟分配收集后得到的关键字序列是（ ）。

- A. 007, 110, 119, 114, 911, 120, 122
- B. 007, 110, 119, 114, 911, 122, 120

C. 007, 110, 911, 114, 119, 120, 122

D. 110, 120, 911, 122, 114, 007, 119

24. 【2021-10】设数组 $S[] = \{93, 946, 372, 9, 146, 151, 301, 485, 236, 327, 43, 892\}$ ，采用最低位优先(LSD)基数排序将S排列成升序序列。第1趟分配、收集后，元素372之前、之后紧邻的元素分别是（ ）。

A. 43, 892

B. 236, 301

C. 301, 892

D. 485, 301

(十一)外部排序

25. 【2016-11】对 10 TB 的数据文件进行排序，应使用的方法是（ ）。

A. 希尔排序

B. 堆排序

C. 快速排序

D. 归并排序

26. 【2019-11】设外存上有120个初始归并段，进行12路归并时，为实现最佳归并，需要补充的虚段个数是（ ）。

A. 1

B. 2

C. 3

D. 4

27. 【2024-11】在外部排序中，利用败者树对初始为升序的归并段进行多路归并，败者树中记录“冠军”的结点保存的是

A. 最大关键字

B. 最小关键字

C. 最大关键字所在归并段号

D. 最小关键字所在归并段号

(十二)排序算法的分析和应用

28. 【2009-10】若数据元素序列11, 12, 13, 7, 8, 9, 23, 4, 5是采用下列排序方法之一得到的第二趟排序后的结果，则该排序算法只能是（ ）。

A. 起泡排序

B. 插入排序

C. 选择排序

D. 二路归并排序

29. 【2010-11】对一组数据 $\langle 2, 12, 16, 88, 5, 10 \rangle$ 进行排序，若前三趟排序结果如下：

第一趟排序结果：〈2,12,16,5,10,88〉

第二趟排序结果：〈2,12,5,10,16,88〉

第三趟排序结果：〈2,5,10,12,16,88〉

则采用的排序方法可能是（ ）。

- A. 起泡排序
- B. 希尔排序
- C. 归并排序
- D. 基数排序

30. 【2012-10】在内部排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列排序方法中，每一趟排序结束都至少能够确定一个元素最终位置的方法是（ ）。

- I. 简单选择排序
 - II. 希尔排序
 - III. 快速排序
 - IV. 堆排序
 - V. 二路归并排序
- A. 仅I、III、IV
 - B. 仅I、III、V
 - C. 仅II、III、IV
 - D. 仅III、IV、V

31. 【2015-09】下列排序算法中，元素的移动次数与关键字的初始排列次序无关的是（ ）。

- A. 直接插入排序
- B. 起泡排序
- C. 基数排序
- D. 快速排序

32. 【2017-11】下列排序方法中，若将顺序存储更换为链式存储，则算法的时间效率会降低的是（ ）。

- I. 插入排序
 - II. 选择排序
 - III. 起泡排序
 - IV. 希尔排序
 - V. 堆排序
- A. 仅I、II
 - B. 仅II、III
 - C. 仅III、IV
 - D. 仅IV、V

33. 【2019-07】选择一个排序算法时，除算法的时空效率，下列因素中，还需要考虑的是（ ）。

- I.数据的规模
- II.数据的存储方式
- III.算法的稳定性
- IV.数据的初始状态

- A.仅 III
- B.仅 I、II
- C.仅 II、III、IV
- D. I、II、III、IV

34. 【2020-11】对大部分元素已有序的数组进行排序时，直接插入排序比简单选择排序效率更高，其原因是（ ）。

- I. 直接插入排序过程中元素之间比较次数更少
- II. 直接插入排序过程中所需要的辅助空间更少
- III. 直接插入排序过程中元素的移动次数更少

- A. 仅I
- B. 仅III
- C. 仅I、II
- D. I、II和III

35. 【2022-11】对数据进行排序时，若采用直接插入排序而不采用快速排序，则可能的原因是（ ）。

- I. 大部分元素已有序
- II. 待排序元素数量很少
- III. 要求空间复杂度为 $O(1)$
- IV. 要求排序算法是稳定的

- A. 仅I、II
- B. 仅III、IV
- C. 仅I、II、IV
- D. I、II、III、IV

36. 【2023-10】下列排序算法中，不稳定的的是（ ）。

- I. 希尔排序
- II. 归并排序
- III. 快速排序
- IV. 堆排序
- V. 基数排序

- A. 仅 I 和 II
- B. 仅 II 和 V
- C. 仅 I、III 和 IV
- D. 仅 III、IV 和 V

